

Test_Lab0 实验报告

——陈思源 2026 年 9 月 27 日

实验任务一

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

答：没有，尽管自己之前学了 C 语言和 python，但由于高中几乎没接触过什么编程，所以也没有什么编程开发的经历。

2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

“暂存-提交”模式的关键在于可以对某一次修改进行提交，不仅不会覆盖编程人员对代码的覆盖，还能简化每次提交的代码，提高了代码的可读性，有利于多人同时编程和开发。

3. git branch 和 git branch -a 的区别是什么？查阅资料并回答。

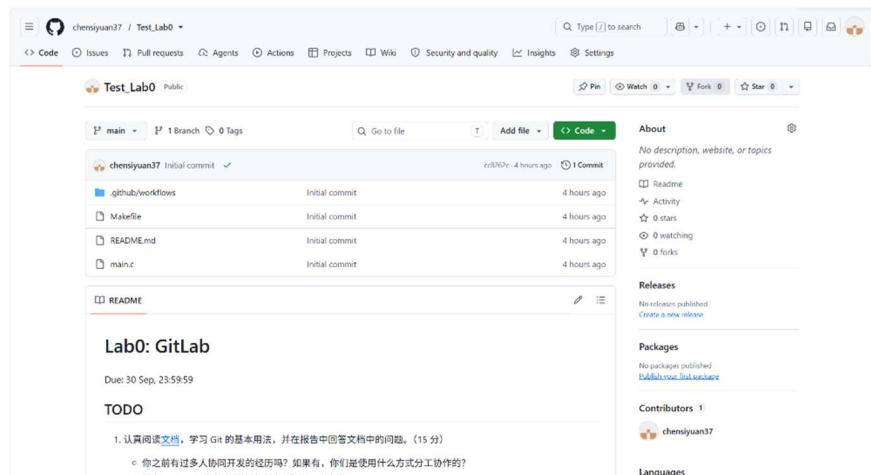
git branch 会列出本地所有的分支，当前所在分支前会有一个 * 号标记。它反映的是本地仓库的分支状态。

git branch -a: 其中-a 是--all 的缩写。它会列出本地分支以及远程跟踪分支 (Remote-tracking branches)，这些远程分支通常以 remotes/origin/ 等前缀开头，代表的是上次与远程仓库通信时远程分支的状态。

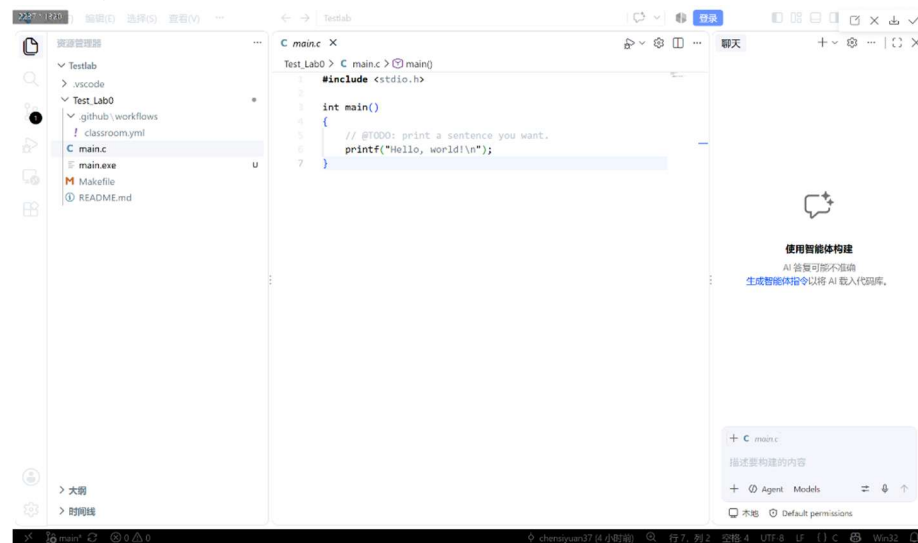
实验任务二

实验步骤：

1. 建立自己的模版仓库，名为 Test_Lab0。



2. 将模版仓库通过 `git clone` + SSH 密钥的方式复制到本地文件里，并用 VS code 打开。



3. 完成 “TODO” 内容



4. 点击右方加号，现已变为 A，即执行 `git add main.c`



5. 输入更改意图，并提交，即 `git commit -m "finish TODO"`



6. 保存后并在终端运行

```
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ make
./main
make clean
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
Hello, world!
Practice makes perfect!
rm -f main.o main
```

实验任务三

1. 阅读了“Commit Message 规范”和“语义化版本”这两篇文章，其大致内容如下：

(1) 前者主要讲了如何统一 Commit message 格式，让工具自动生成规范 Change log，具体如下：

目的：规范 Git 提交说明，便于阅读、过滤，并能自动生成 Change log。

格式：采用 Angular 规范，分 Header、Body、Footer。

Header（必填）：type(scope): subject

type（常用）：feat、fix、docs、style、refactor、test、chore

工具：

Commitizen：交互式生成规范提交信息

validate-commit-msg：检查提交信息是否合格

自动生成 Change log：用 conventional-changelog，按 feat、fix、BREAKING CHANGE 分类生成日志。

(2) 后者 Semantic Versioning（语义化版本，简称 SemVer）2.0.0 是一套为软件版本号制定明确含义的规范，旨在解决依赖管理中的“依赖地狱”问题，其通过一套清晰的规则，让版本号本身就能传达底层代码的变化性质，从而使依赖管理变得更加可靠和自动化。具体方面如下：

版本号格式：

标准的版本号采用 X.Y.Z 格式，其中 X、Y、Z 均为非负整数，且数字前禁止补零。

X：主版本号（MAJOR）

Y：次版本号（MINOR）

Z: 修订号 (PATCH)

递增规则:

版本号的递增遵循以下三条核心规则:

1. 主版本号: 当你做了不兼容的 API 修改时递增。
2. 次版本号: 当你做了向下兼容的功能性新增时递增。
3. 修订号: 当你做了向下兼容的问题修正时递增。

扩展信息:

在 X.Y.Z 之后, 可以附加先行版本号 (如 -alpha.1) 和版本编译元数据 (如 +20130313144700) 作为延伸。

关键规则:

标记版本号的软件发行后, 禁止改变该版本的内容; 任何修改都必须以新版本发布。

必须先定义好公共 API, 版本号才能有效传达修改的含义。

初始开发阶段:

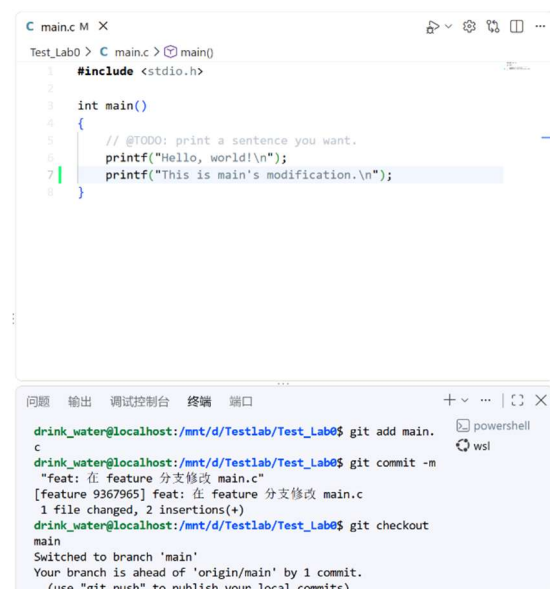
在 0.y.z 阶段, API 不稳定, 可以快速迭代。当软件被用于正式环境或已有稳定的公共 API 时, 才应发布 1.0.0 版本。

2. 所以为什么要学习 Git?

- (1) 每次修改都有记录, 随时能回退
- (2) 开分支随便试, 不怕弄坏主版本
- (3) 多人协作能合并代码, 冲突也有办法解决
- (4) 能参与开源项目, 找工作也常被问到

实验任务四

1. 新建 feature 分支并修改。



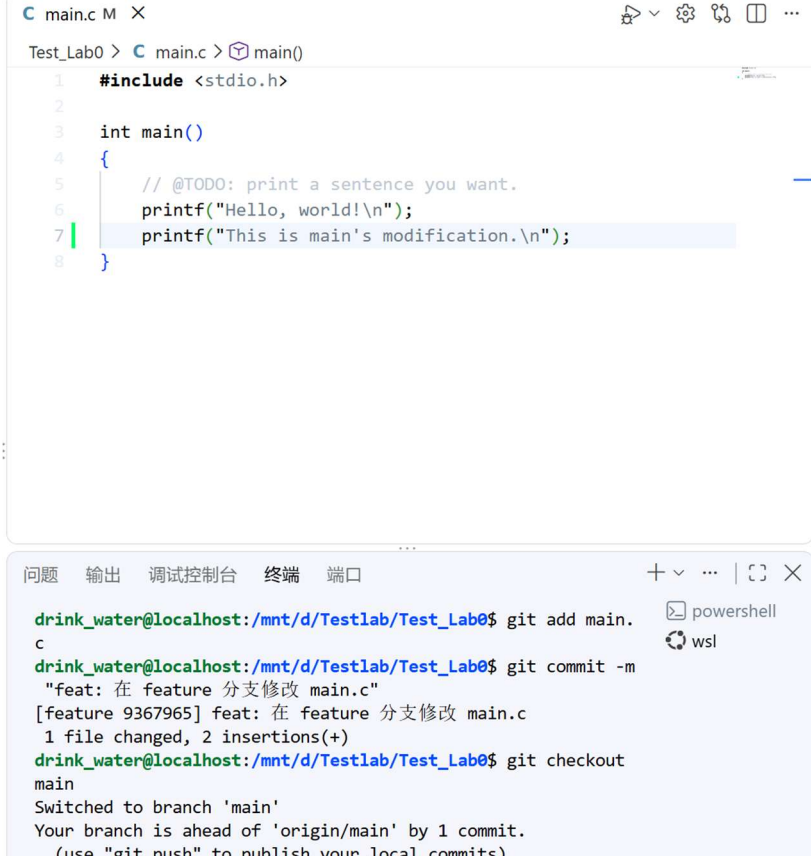
```
C main.c M X
Test_Lab0 > C main.c > main()
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("Hello, world!\n");
7     printf("This is main's modification.\n");
8 }

drink_water@localhost:/mnt/d/TestLab/Test_Lab0$ git add main.c
c
drink_water@localhost:/mnt/d/TestLab/Test_Lab0$ git commit -m
"feat: 在 feature 分支修改 main.c"
[feature 9367965] feat: 在 feature 分支修改 main.c
1 file changed, 2 insertions(+)
drink_water@localhost:/mnt/d/TestLab/Test_Lab0$ git checkout
main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
```

2. 保存、暂存并提交。

```
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git config --
global user.name "chensiyuan"
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git config --
global user.email "3070254799@qq.com"
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git add main.
c
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git commit -m
"feat: 在 feature 分支修改 main.c"
[feature 9367965] feat: 在 feature 分支修改 main.c
1 file changed, 2 insertions(+)
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$
```

3. 回到 main 分支，进行不一样的修改。保存、暂存并提交。



The screenshot shows a code editor with a file named `main.c`. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("Hello, world!\n");
7     printf("This is main's modification.\n");
8 }
```

Below the code editor is a terminal window. The terminal shows the following commands and output:

```
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git add main.
c
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git commit -m
"feat: 在 feature 分支修改 main.c"
[feature 9367965] feat: 在 feature 分支修改 main.c
1 file changed, 2 insertions(+)
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git checkout
main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git add main.
c
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git commit -m
"fix: 在 main 分支修改 main.c"
[main d462c2c] fix: 在 main 分支修改 main.c
1 file changed, 1 insertion(+)
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$
```

4. 在终端执行 git merge 后遇到冲突。

The image shows a code editor window titled "main.c 4,!" and a terminal window below it. The code editor displays a C program with a merge conflict in the `main` function. The conflict is between the `HEAD` (current changes) and the `feature` branch (incoming changes). The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("Hello, world!\n");
7     <<<<<<< HEAD (当前更改)
8     printf("This is main's modification.\n");
9     =====
10    printf("Practice makes perfect!\n");
11    printf("This is feature's modification.\n")
12    >>>>>>> feature (传入的更改)
13 }
```

The terminal window shows the following commands and output:

```
c
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git commit -m
"fix: 在 main 分支修改 main.c"
[main d462c2c] fix: 在 main 分支修改 main.c
1 file changed, 1 insertion(+)
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git merge fea
ture
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the res
ult.
```

5. 删掉“<<<<<<< HEAD、=====、>>>>>>> feature”等特殊符号，然后完成合并提交。

The terminal window shows the following commands and output:

```
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git add main.
c
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ git commit -m
"merge: 解决冲突"
[main 745dc23] merge: 解决冲突
```

6. 总的修改过程以及最后的代码运行结果。



```
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ make
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ ./main
Hello, world!
This is main's modification.
Practice makes perfect!
This is feature's modifacation.
drink_water@localhost:/mnt/d/Testlab/Test_Lab0$ make clean
rm -f main.o main
```

思考：如何修改 main.c 会出现冲突？

在 feature 分支和 main 分支上修改同一行且修改内容不同（均保存）时，最后合并会出现冲突。